

EECE 7205. Fundamentals of Computer Engineering.

Fall 2026

Dr. Fatema Nafa

Assignment #01

Before beginning your work, read all sections carefully. For every problem, you must follow the same problem-solving steps demonstrated in class: understand the problem, identify the inputs and outputs, develop pseudocode, analyze the algorithm, implement the solution in C++, and test it. The problems progress from intermediate to advanced difficulty and are modeled after common software-engineering interview questions.

Assignment Objectives:

- Apply sorting techniques to solve realistic software-engineering interview problems.
- Use C++ vectors, strings, structs, hash maps, heaps, and custom comparison functions.
- Select an appropriate algorithm by analyzing time and space complexity.
- Handle duplicates, ties, empty input, and other important edge cases.
- Explain and test solutions clearly, as expected during a technical interview.

Required Problem Solving Process:

For each problem, complete the following steps in order. Your report and C++ program must show that you followed the same process used in class.

- 1. Understand the problem:** Identify the goal, inputs, outputs, constraints, assumptions, and important edge cases.
- 2. Design the solution:** Select an appropriate algorithm or data structure and briefly explain why it fits the problem.
- 3. Write pseudocode:** Write clear, language-independent pseudocode before writing the C++ implementation. Use the same pseudocode format and level of detail demonstrated in class.
- 4. Explain correctness:** Briefly explain how the algorithm produces the required result and handles duplicates, ties, empty inputs, and other relevant edge cases.
- 5. Analyze complexity:** Determine and justify the time complexity of your algorithm using Big-O notation. Include best-case, average-case, and worst-case complexity when they differ. Also state the space complexity.
- 6. Implement in C++:** Translate the pseudocode into a clear, functional C++ solution. Use meaningful names, functions, consistent formatting, and comments for non-obvious logic.
- 7. Test the solution:** Use the provided example and at least two additional test cases for each problem, including at least one edge case. Include screenshots that clearly show the selected problem, input, and output.

Submission Requirements:

Submit exactly two items in one GitHub repository:

1. PDF report: Create the report in Overleaf and submit the compiled PDF file. For every problem, include the problem analysis, pseudocode, selected approach, correctness explanation, time and space complexity, test cases, and execution screenshots. Suggested filename: Assignment01_LastName.pdf.

2. cpp folder: Include one C++ program that contains solutions to all problems in this assignment. The program must display a menu that allows the user to choose which problem to run. Organize each solution in a separate function and keep all required C++ source files inside the cpp folder. Suggested main filename: Assignment01_LastName.cpp.

GitHub submission: Place the PDF report and the cpp folder in the same repository, push the final versions to GitHub, and verify that the repository and files can be opened. The repository must be public or shared with the instructor.

Canvas submission: **Submit the GitHub repository link on Canvas before the deadline.** Make sure the link points to the correct repository and works without requesting access.

Tasks:**Problem 1:**

Description: Intervals appear in calendars, resource reservations, timelines, and scheduling systems. Overlapping intervals can be combined into a smaller set of non-overlapping intervals.

Question: Given an array of intervals [start, end], sort the intervals by starting value and merge every overlapping interval. Return the resulting non-overlapping intervals.

Problem 2:

Description: A manufacturing line labels items with three categories represented by 0, 1, and 2. The items must be grouped in category order.

Given an array containing only 0, 1, and 2, sort it in place. First implement a counting-based solution. Then implement the one-pass Dutch National Flag algorithm without using `std::sort()`.

Problem 3:

Description: A custom ordering is sometimes required when the normal numeric order does not produce the desired combined result.

Given a vector of nonnegative integers, arrange them so that their concatenation forms the largest possible number. Return the answer as a string.

Problem 4: Group Anagrams

Description: Anagrams contain the same characters with the same frequencies but in a different order. Grouping them is useful in text indexing and search.

Given a vector of strings, group the anagrams together. The order of groups does not matter, but words within each group should retain their original input order.

Problem 5:

Description: Calendar software must determine whether a person or resource has been assigned to overlapping meetings.

Given meeting intervals [start, end], determine whether one person can attend every meeting. Meetings that begin exactly when other ends are allowed.

Problem 6:

Navigation, robotics, and spatial systems often need to identify the nearest locations or sensor readings.

Given points (x, y) and an integer k , return the k points closest to the origin. Compare a sorting solution with a heap-based solution.

Problem 7:

Description: Monitoring and analytics systems often rank events, error codes, or queries by frequency.

Question: Given an integer array and an integer k , return the k most frequent values. If frequencies tie, return the smaller value first.

Problem 8:

Description: Distributed systems may produce multiple individually sorted streams that must be combined into one ordered result.

Question: Given k sorted integer arrays, merge them into one sorted array. Design a solution that does not repeatedly sort the complete output.

Problem 9:

Description: Ranking systems often need one order statistic without paying the cost of fully sorting a large dataset.

Question: Given an unsorted integer array and k , return the k th-largest element. Duplicate values count as separate positions.

Problem 10: Count Inversions

Description: An inversion measures how far an array is from sorted order and is used in ranking analysis and sequence comparison.

Question: Count pairs (i, j) for which $i < j$ and $\text{values}[i] > \text{values}[j]$. Develop an efficient solution by modifying Merge Sort.

Example Input	[2, 4, 1, 3, 5]
Expected Output	3
Requirements	A brute-force $O(n^2)$ solution may be used for validation, but the submitted primary solution must run in $O(n \log n)$ time. Use a 64-bit integer for the inversion count.

Notes:

During an interview, clearly state assumptions, discuss edge cases, and explain why your algorithm is correct before focusing on implementation details. If you encounter any issues while completing this assignment, please reach out for assistance.